

The C++ programming language

December 22, 2025
Ramasamy Kandasamy

Contents

1	Language	2	7	Devtools	13
1.1	Pre-processor	2	7.1	G++	13
1.2	Expressions	2	7.2	GDB	13
1.3	Declarations	3			
1.4	Initialization	3			
1.5	Functions	4			
1.6	Statements	4			
1.7	Classes	5			
1.8	Templates	6			
1.9	Miscellaneous	7			
2	Utils	7			
2.1	Data Elements	7			
2.2	Memory management in C++	7			
2.3	limits	7			
3	Iterators	7			
3.1	Iterator Categories	7			
3.2	Reverse Iterators	7			
3.3	Iterator Utilities	8			
3.4	Iterator Adaptors	8			
4	Algorithms	8			
4.1	Non-Mutating Algorithms	8			
4.2	Mutating Algorithms	8			
4.3	Sorting & Set Operations	9			
4.4	Numeric Algorithms	9			
4.5	Compare Function	9			
5	Containers	10			
5.1	Common Methods	10			
5.2	Sequential Containers	10			
5.3	Associative Containers	10			
5.4	Container Adapters	10			
6	Libraries	11			
6.1	Input output	11			
6.2	cin and cout	12			
6.3	Miscellaneous	13			
6.4	String	13			

1. Language

1.1. Pre-processor

File Inclusion:

- `#include <filename>` — Search system include paths.
- `#include "filename"` — Search current directory first, then system paths.

Macro Definitions:

- `#define NAME replacement`
- `#define FUNC(arg1, arg2) expression`
Eg: `#define MAX(A,B) ((A) > (B) ? (A) : (B))`
- `#undef NAME` — Remove definition of NAME.

Macro Operators:

- `#` (stringification) — Converts argument to string literal.
Eg: `#define STR(x) #x` ⇒ `STR(hello)` expands to `"hello"`
- `##` (token pasting) — Concatenates tokens.
Eg: `#define CONCAT(a,b) a##b` ⇒ `CONCAT(x,y)` expands to `xy`

Conditional Compilation:

- `#if constant-expression / #elif / #else / #endif`
- `#ifdef NAME` — True if NAME is defined.
- `#ifndef NAME` — True if NAME is not defined.
- `defined(NAME)` — Can be used in `#if` expressions.

Example header guard:

```
#ifndef HEADER_FILE_H
#define HEADER_FILE_H
// ... declarations ...
#endif
```

- `#pragma once` — Modern single-inclusion guard (non-standard but widely supported).

Diagnostics:

- `#error "message"` — Emit compile-time error.
- `#warning "message"` — Emit warning (compiler extension).
- `#line N "file"` — Change reported line number and filename in diagnostics.

Predefined Macros:

<code>__FILE__</code>	Current source file name (string).
<code>__LINE__</code>	Current line number (integer).
<code>__DATE__</code>	Compilation date as "Mmm dd yyyy".
<code>__TIME__</code>	Compilation time as "hh:mm:ss".
<code>__cplusplus</code>	C++ standard version (e.g., 201703L for C++17).
<code>__func__</code>	Current function name (C++11, not technically a macro).
<code>__STDC_HOSTED__</code>	1 if hosted implementation, 0 if freestanding.

1.2. Expressions

Value Categories

<code>lvalue</code>	Has name. Can take address.
<code>prvalue</code>	Temporary value. No name. <i>Guaranteed copy elision.</i>
<code>xvalue</code>	Expiring. Eg: <code>std::move(x)</code> .
<code>glvalue</code>	<code>lvalue</code> $\cup$ <code>xvalue</code> .
<code>rvalue</code>	<code>prvalue</code> $\cup$ <code>xvalue</code> .
<code>L-value reference</code>	Refers to an <code>lvalue</code> . Eg: <code>int& ref = var;</code>
<code>R-value reference</code>	Refers to an <code>rvalue</code> . Eg: <code>int&& rref = 5;</code>

Exceptions:

- R-value can bind to `const` l-value reference.
Eg: `const int& x = 42;`
- String literals are l-values.

Literals

Boolean	<code>true</code>	
	<code>false</code>	
Integer	<code>42</code>	Decimal.
	<code>4'200'000</code>	Decimal with separators.
	<code>0b101010</code>	Binary.
	<code>052</code>	Octal.
	<code>0x2a</code>	Hexadecimal.
Float	<code>3.14</code>	Double.
	<code>3.14f</code>	Float.
	<code>6.626e-34</code>	<code>xey</code> → $x \times 10^y$
Character	<code>'a'</code>	Ordinary literal.
	<code>'\n'</code>	Using escape seq.
	<code>'\13'</code>	
	<code>u8'a'</code>	UTF-8.
String	<code>"Hello"</code>	String literal.
Pointer	<code>nullptr</code>	

NOTE: Literals are prvalues.

Exception: String literals are lvalues.

std::initializer_list

Constructed automatically from braced-init-list.

Eg:

```
Foo foo({1, 2, 3});
print_list({1, 2, 3});
```

Passed as argument to functions.

Eg:

```
void print_list(std::initializer_list<int> list);
Foo(std::initializer_list<int> list);
```

Access elements using iterators or range-based for loop.

Arithmetic and logical operation

	Operators	Associativity
1	<code>() [] -> .</code>	left to right
2	<code>! ~ ++ -- + - * & (type) sizeof</code>	right to left
3	<code>* / %(remainder)</code>	left to right
4	<code>+ -</code>	left to right
5	<code><< >></code>	left to right
6	<code>< <= > >=</code>	left to right
7	<code>== !=</code>	left to right
8	<code>&</code>	left to right
9	<code>^</code>	left to right
10	<code> </code>	left to right
11	<code>&&</code>	left to right
12	<code> </code>	left to right
13	<code>?:</code>	left to right
14	<code>= += -= *= /= %= &= ^= = <<= >>=</code>	right to left
15	<code>,</code>	left to right

Selected operators

- `>>` Bitwise right shift operator.
`a = b >> 1` equivalent to dividing `b` by 2.
- `<<` Bitwise left shift operator.
`a = b << 1` equivalent to multiplying `b` by 2.
- `?:` Ternary conditional operator.
`x = (condition)? expr1: expr2;`
`expr1` → `condition == true`.
`expr2` → `condition == false`.
- `~` Bitwise negation.
- `^` Bitwise XOR.

Type casting

<code>const_cast</code>	Modifies <code>const</code> /volatile qualifiers.
<code>static_cast</code>	Modifying a <code>const</code> object leads to UB. Compile-time downcast. Not safe.
<code>dynamic_cast</code>	Safe downcast. Eg: <code>Base* b = static_cast<Base*>d;</code> when invalid <code>b</code> is <code>nullptr</code> . Eg: <code>Base& b = static_cast<Base&>d;</code> when invalid throws <code>std::bad_cast</code> .
<code>reinterpret_cast</code>	

1.5. Functions

- **Definition :** `T foo(args) {...}`.
- **Declaration :** `T foo(args);`.
- **Variadic Arguments:** `T foo(int n, ...)`.
C-style. Not safe. Use variadic templates.

The main() function

```
main(int argc, char *argv[]) {  
    ...  
}  
argc      No. of args including the program name.  
argv[0]    Pointer to executable name.  
argv[i]    Pointer to ith argument.
```

Lambda function

Usage: `[<capture>](<params>)-><return-type>{<body>}`

`<capture>` Specifics

`[]` No variables captured.

`[=]` All variables captured by value.

`[&]` All variables captured by reference.

`[x, &y]` x captured by value, y by reference.

`[=, &y]` All variables except y by value, y by reference.

NOTE: Not specifying return type is not compile time error and after compiler can deduce it. But, it is good practice to specify it, especially for complex lambdas.

Eg:

```
// Sort descending using lambda  
sort(nums.begin(), nums.end(), [](int a, int b) -> bool {  
    return a > b;  
});
```

Passing functions as arguments

1. Function pointers.

Eg:

```
int foo(int a, int b, int (*bar)(int, int)) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Doesn't work with lambdas with captures.

2. `std::function`

```
int foo(int a, int b, std::function<int(int, int)> bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

3. Templates

```
template <typename bar>  
int foo(int a, int b, bar bar) {  
    return bar(a, b);  
}  
// Function call.  
foo(5, 3, add);
```

Zero overhead

1.6. Statements

If-else	<pre>if (<i>expression</i>) <i>statement</i> else if (<i>expression</i>) <i>statement</i> : : else <i>statement</i></pre>
Switch	<pre>switch (<i>expression</i>) { case <i>const-expr</i>: <i>statements</i>; break; case <i>const-expr</i>: <i>statements</i>; break; default: <i>statements</i>; break; } Without break statement the execution will <i>fall through</i> all the cases. This can be used as: switch (<i>expression</i>) { case 1: case 2: case 3: group = 0; break; case 4: case 5: group = 1; break; default: group = -1; break; }</pre>
While	<pre>while (<i>expression</i>) <i>statement</i></pre>
For	<pre>for (expr1; expr2; expr3) <i>statement</i> This for loop is equivalent to: expr1; while (expr2) { <i>statements</i>; expr3; }</pre>
Do While	<pre>do <i>statement</i> while (<i>expression</i>);</pre>
Break	break : Exit the the loop or control. Works for while, for, do while and switch

Continue	continue: Continue next iteration. Works for while, for, do while If switch is within a loop and if continue is used inside switch and if it is executed, then the loop skips to next loop.
Goto and label	Example: if (disaster) { goto error; } ... error: { statement }

Shorter alternative for if-else statement:
x = <cond-expr> ? <out-if-true> : <out-if-false>;
Eg: char is_pos = (n > 0) ? 'y' : 'n';

1.7. Classes

Eg:

```
class Point {
public:
    Point(int x, int y) : x_(x), y_(y) {}
private:
    int x_;
    int y_;
};
```

Inheritance Behaviour

class <derived> : <type> <base>.

Eg: class Foo : public Bar.

Simple rule

1. Private members are never inherited.
2. derived-type = min(base-type, inheritance-type).

Member / Inheritance	Public	Protected	Private
Public	Public	Protected	Private
Protected	Protected	Protected	Private
Private	NA	NA	NA

Members Variables

- this - pointer to the current object.
- static - shared across all objects of the class.

Member Functions

Rule-of-5 members

1. Copy Ctor

```
// Pass by ref to avoid recursion.
Vec::Vec(const Vec &other) {
    n_ = other.n_;
    data_ = new int[n_];
    for (int i = 0; i < n_; i++) {
        data_[i] = other.data_[i];
    }
}
```

2. Move Ctor

```
Vec::Vec(Vec &&other) noexcept {
    n_ = other.n_;
    data_ = other.data_;
    other.n_ = 0;
    other.data_ = nullptr;
}
```

3. Copy Assignment Operator

```
Vec& Vec::operator=(const Vec &other) {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = new int[n_];
        // Deep copy.
        for (int i = 0; i < n_; i++) {
            data_[i] = other.data_[i];
        }
    }
    return *this; // To permit chained assignments.
}
```

4. Move Assignment Operator

```
Vec& Vec::operator=(Vec &&other) noexcept {
    if (this != &other) { // Self-assignment check.
        delete[] data_; // Free existing resource.
        n_ = other.n_;
        data_ = other.data_;
        other.n_ = 0;
        other.data_ = nullptr;
    }
    return *this; // To permit chained assignments.
}
```

5. Destructor

```
Vec::~Vec() {
    delete[] data_;
}
```

Others

6. Default Ctor Eg: Foo::Foo()

7. Converting Ctor

Ctor with single argument or multiple arguments with default values.

```
Say: Foo::Foo(const Bar&);

Foo f = bar; // Implicit conversion from Bar to Foo.

explicit Foo(const Bar&); // Prevents implicit conversion.
```

8. Static methods

Belong to class, not object.
No this pointer.

Compilers provided default implementation for 1-6 methods.

- **delete:** Prevents default generation.
- **default:** Makes the intent clear.
Forces default generation?
- **explicit:** Prevents implicit conversions for converting ctors.

Virtual Methods and Abstract Class

- **Virtual method:** Eg: virtual void Animal::speak();
- **Pure virtual method:** Eg: virtual void Animal::speak() = 0;
- **Abstract class:** Class with at least one pure virtual method. Cannot instantiate objects of abstract class.

Virtual vs normal methods:

```
Animal* a = new Dog();
a->speak();
```

- **speak** is virtual: Dog's speak() called.
- **speak** is normal: Animal's speak() called.

override & final

Specifiers at the end. Relative order between them doesn't matter.

override:

Tells a method is meant to override a base class method.

Compiler error if it doesn't.

Eg:

```
Base: void Animal::speak() const;
Derived: void Dog::speak(); // Compiles.
Unexpected runtime behaviour:
a->speak(); // Base class speak.
Derived w/ override:
void Dog::speak() override; // Compiler error.
```

final:

Method not supposed to be overridden in derived classes.

Compiler error if it is.

Eg:

```
Base: void Dog::speak() override final;
Derived:
void BorderCollie::speak() override; // Compiler error.
```

1.8. Templates

Template Parameter Kinds

Type	typename T or class T Accepts any type as argument.
Non-type	int N, size_t N, auto N, etc. Accepts compile-time constant values. Types: integral, pointers, references, nullptr_t, enums, float (C++20).
Template template	template <typename> class C template <typename> typename C (C++17+) Accepts template as argument.

Template Parameter Syntax

Basic	typename T int N template <typename> class C
With default	typename T = int int N = 10 template <typename> class C = std::vector
Variadic	typename... Args int... Ns template <typename> class... Containers

NOTE: **typename** vs **class** for type parameters are interchangeable. **typename** is preferred (more explicit). Also, **typename** has another use: disambiguating dependent types. Eg: **typename T::value_type x**;

Template Specialization

Full/Explicit Specialization

Provide custom implementation for specific type(s).

Eg:

```
// Primary template
template <typename T>
class MyClass { /*...*/ };

// Full specialization for bool
template <>
class MyClass<bool> { /*...*/ };
```

Partial Specialization

Specialize for a subset of types. **Only for class templates, not functions.**

Eg:

```
// Primary template
template <typename T, typename U>
class Pair { /*...*/ };
```

```
// Partial specialization: both types same
template <typename T>
class Pair<T, T> { /*...*/ };
```

```
// Partial specialization: pointer types
template <typename T, typename U>
class Pair<T*, U*> { /*...*/ };
```

Full specialization	template <> class C<int> {...} All parameters specified. Works for classes and functions.
Partial specialization	template <typename T> class C<T*> {...} Some parameters remain generic. Only for class templates.

Parameter Packs

Syntax

Declaration	typename... Args int... Ns
Expansion	Args... expands to T1, T2, T3, ... func(args)... expands to func(a1), func(a2), ... sizeof...(Args) gives pack size.

Example: Compile-time Fibonacci sequence

```
// Fibonacci calculation
template <int N>
struct Fib {
    static constexpr int value =
        Fib<N-1>::value + Fib<N-2>::value;
};

template <>
struct Fib<0> { static constexpr int value = 0; };
template <>
struct Fib<1> { static constexpr int value = 1; };

// Variadic template to hold sequence
template <int... Ns>
struct FibSeq {
    static constexpr int values[] = { Fib<Ns>::value... };
};

// Usage
constexpr int fibs[] = FibSeq<0, 1, 2, 3, 4, 5>::values;
// fibs = {0, 1, 1, 2, 3, 5}
```

Pack expansion Fib<Ns>::value... expands to Fib<0>::value, Fib<1>::value, ..., Fib<5>::value

1.9. Miscellaneous

2. Utils

2.1. Data Elements

Pair

Eg: `pair<int, int> x = 1,2;`
`x.first` Returns 1.
`x.second` Returns 2.

Tuple

Fixed length collection of heterogenous values.

Eg: `tuple<int, string, double> t;`

Operations related to tuple

`get<i>` Return i^{th} value of t. Eg: `get<0>(t);`.
`make_tuple()` Returns tuple out of individual values.
Eg: `make_tuple(5, "Paul", 23.4)`.
`tie()` Unpacking tuple.
Eg:
`tie(n, s, x) = t;`
`tie(n, std::ignore, x) = t;`

Bit set

Definiton

`bitset<size> s;`

Examples:

`bitset<5> s; bitset<5> s(string("1100110"));`

Operations on Bit set:

`a & b` Bitwise AND.
`a | b` Bitwise OR.
`a ^ b` Bitwise EXOR.
`a` Bitwise negation.

2.2. Memory management in C++

- `void* operator new(size)`
Eg: `myClass* p1 = new myClass;`
`myClass* p1 = new(sizeof(myClass));`
- `void delete(void* ptr)`
Eg: `delete p1; delete(p1);`

2.3. limits

Examples:

- `numeric_limits<int>::min()`
- `numeric_limits<int>::max()`
- `numeric_limits<double>::min()`
- `numeric_limits<double>::infinity()`

3. Iterators

3.1. Iterator Categories

Input Iterator

Read-only, single-pass, forward-moving.
Supports: `*it` (read), `++it`, `it++`, `==`, `!=`.
Example: `istream_iterator`.

Output Iterator

Write-only, single-pass, forward-moving.
Supports: `*it = val`, `++it`, `it++`.
Example: `ostream_iterator`, `back_inserter`.

Forward Iterator

Multi-pass, forward-moving, read/write.
Supports: All input iterator ops + can traverse multiple times.
Example: `forward_list::iterator`.

Bidirectional Iterator

Like forward iterator but can move backward.
Supports: All forward iterator ops + `--it`, `it--`.
Example: `list::iterator`, `set::iterator`, `map::iterator`.

Random Access Iterator

Full pointer-like arithmetic.
Supports: All bidirectional ops + `it[n]`, `it + n`, `it - n`, `it += n`, `it -= n`, `<`, `>`, `<=`, `>=`.
Example: `vector::iterator`, `deque::iterator`, raw pointers.

Random Access Examples

`v.begin() + 5` Iterator to 6th element.
`it[3]` Access element 3 positions ahead.
`it2 - it1` Distance between iterators.
`it1 < it2` Compare positions.

3.2. Reverse Iterators

Traverse containers backward. Available on bidirectional+ containers.

Getting Reverse Iterators

`c.rbegin()` Reverse iterator to last element.
`c.rend()` Reverse iterator before first element.
`c.crbegin()` Const reverse begin.
`c.crend()` Const reverse end.

Usage Example

```
vector<int> v = {1, 2, 3, 4, 5};
for (auto it = v.rbegin(); it != v.rend(); ++it)
    cout << *it; // prints: 54321
```

Base Iterator

`rit.base()` returns forward iterator pointing one past `rit`.
If `rit` points to element at position i , `rit.base()` points to $i + 1$.

3.3. Iterator Utilities

<code>advance(it, n)</code>	Move iterator by <code>n</code> positions. Works on all iterators.
<code>distance(first, last)</code>	Number of increments from first to last.
<code>next(it, n=1)</code>	Return iterator <code>n</code> positions ahead (C++11).
<code>prev(it, n=1)</code>	Return iterator <code>n</code> positions back (C++11).
<code>begin(c) / end(c)</code>	Free functions for any container or array.
<code>cbegin(c) / cend(c)</code>	Const versions (C++14).

3.4. Iterator Adaptors

<code>back_inserter(c)</code>	Output iterator that calls <code>push_back</code> .
<code>front_inserter(c)</code>	Output iterator that calls <code>push_front</code> .
<code>inserter(c, it)</code>	Output iterator that calls <code>insert</code> at position.
<code>make_reverse_iterator(it)</code>	Create reverse iterator from forward (C++14).
<code>make_move_iterator(it)</code>	Iterator that moves instead of copies (C++11).

4. Algorithms

`#include <algorithm>`, `#include <numeric>`

All algorithms work on iterator ranges `[first, last)`. Optional `comp` is a binary predicate; `pred` is a unary predicate.

4.1. Non-Mutating Algorithms

Finding Elements

- `It find(first, last, val)` – Returns iterator to first element equal to `val`, or `last` if not found.
- `It find_if(first, last, pred)` – Returns iterator to first element where `pred` returns true, or `last`.
- `It find_if_not(first, last, pred)` – Returns iterator to first element where `pred` returns false, or `last`.
- `It adjacent_find(first, last, comp)` – Returns iterator to first of two consecutive equal elements, or `last`.

Counting

- `size_t count(first, last, val)` – Returns number of elements equal to `val`.
- `size_t count_if(first, last, pred)` – Returns number of elements where `pred` is true.

Comparing Ranges

- `bool equal(f1, l1, f2, comp)` – Returns true if all elements in both ranges are equal.
- `pair<It,It> mismatch(f1, l1, f2, comp)` – Returns pair of iterators to first mismatching elements in both ranges.

Searching

- `It search(f1, l1, f2, l2, comp)` – Returns iterator to start of first occurrence of subsequence `[f2,l2)`, or `l1`.
- `It search_n(first, last, n, val, comp)` – Returns iterator to start of first sequence of `n` consecutive elements equal to `val`, or `last`.
- `It find_end(f1, l1, f2, l2, comp)` – Returns iterator to start of *last* occurrence of subsequence, or `l1`.
- `It find_first_of(f1, l1, f2, l2, comp)` – Returns iterator to first element in `[f1,l1)` matching any in `[f2,l2)`, or `l1`.

Binary Search (sorted ranges)

- `bool binary_search(first, last, val, comp)` – Returns true if `val` exists in sorted range.
- `It lower_bound(first, last, val, comp)` – Returns iterator to first element $\geq$ `val`, or `last`.
- `It upper_bound(first, last, val, comp)` – Returns iterator to first element $>$ `val`, or `last`.
- `pair<It,It> equal_range(first, last, val, comp)` – Returns pair: `(lower_bound, upper_bound)`.

Min/Max

- `It min_element(first, last, comp)` – Returns iterator to smallest element, or `last` if empty.
- `It max_element(first, last, comp)` – Returns iterator to largest element, or `last` if empty.
- `pair<It,It> minmax_element(first, last, comp)` – Returns pair: `(min_element, max_element)`.

Predicates on Ranges

- `bool all_of(first, last, pred)` – Returns true if `pred` is true for all elements (or range is empty).
- `bool any_of(first, last, pred)` – Returns true if `pred` is true for at least one element.
- `bool none_of(first, last, pred)` – Returns true if `pred` is false for all elements (or range is empty).

Iteration

- `Func for_each(first, last, func)` – Applies `func` to each element. Returns `func` (useful for stateful functors).

4.2. Mutating Algorithms

Copying

- `It copy(first, last, dest)` – Copies range to `dest`. Returns iterator past last copied element in destination.
- `It copy_if(first, last, dest, pred)` – Copies elements where `pred` is true. Returns iterator past last copied element.
- `It copy_n(first, n, dest)` – Copies `n` elements. Returns iterator past last copied element in destination.
- `It copy_backward(first, last, dest_last)` – Copies range backward ending at `dest_last`. Returns iterator to first copied element.
- `It move(first, last, dest)` – Move-assigns elements to `dest`. Returns iterator past last moved element.

Filling & Generating

- `void fill(first, last, val)` – Assigns `val` to all elements. Returns nothing.
- `It fill_n(first, n, val)` – Assigns `val` to first `n` elements. Returns iterator past last filled element.
- `void generate(first, last, gen)` – Assigns result of calling `gen()` to each element. Returns nothing.
- `It generate_n(first, n, gen)` – Assigns `gen()` to first `n` elements. Returns iterator past last assigned element.

Transforming

- `It transform(first, last, dest, op)` – Applies `op` to each element, stores result in `dest`. Returns iterator past last transformed element.
- `It transform(f1, l1, f2, dest, binop)` – Applies `binop` to pairs from two ranges. Returns iterator past last transformed element.

Replacing

- `void replace(first, last, old, new)` – Replaces all elements equal to `old` with `new`. Returns nothing.
- `void replace_if(first, last, pred, new)` – Replaces elements where `pred` is true. Returns nothing.

Removing

- `It remove(first, last, val)` – Moves elements $\neq$ `val` to front. Returns iterator to new logical end.
- `It remove_if(first, last, pred)` – Moves elements where `pred` is false to front. Returns iterator to new logical end.
- `It unique(first, last, comp)` – Removes consecutive duplicates. Returns iterator to new logical end.

Note: These don't resize; use erase-remove idiom:

```
v.erase(remove(...), v.end());
```

Reordering

- `void reverse(first, last)` – Reverses elements in range. Returns nothing.
- `It reverse_copy(first, last, dest)` – Copies reversed range to `dest`. Returns iterator past last copied element.
- `It rotate(first, middle, last)` – Rotates so `middle` becomes first. Returns iterator to original first element's new position.
- `void shuffle(first, last, rng)` – Randomly shuffles using random generator `rng`. Returns nothing.

Swapping

- `void swap(a, b)` – Swaps values of `a` and `b`. Returns nothing.
- `void iter_swap(it1, it2)` – Swaps elements pointed to by iterators. Returns nothing.
- `It swap_ranges(f1, l1, f2)` – Swaps elements in two ranges. Returns iterator past last swapped element in second range.

4.3. Sorting & Set Operations

Sorting

- `void sort(first, last, comp)` – Sorts in ascending order; $O(n \log n)$. Returns nothing.
- `void stable_sort(first, last, comp)` – Stable sort; preserves order of equal elements. Returns nothing.
- `void partial_sort(first, middle, last, comp)` – Sorts smallest elements into `[first, middle)`. Returns nothing.
- `void nth_element(first, nth, last, comp)` – Partitions so element at `nth` is in sorted position. Returns nothing.
- `bool is_sorted(first, last, comp)` – Returns true if range is sorted.
- `It is_sorted_until(first, last, comp)` – Returns iterator to first element that breaks sorted order, or `last`.

Partitioning

- `It partition(first, last, pred)` – Reorders so elements satisfying `pred` come first. Returns iterator to first element of second group.
- `It stable_partition(first, last, pred)` – Stable partition. Returns iterator to first element of second group.
- `bool is_partitioned(first, last, pred)` – Returns true if all elements satisfying `pred` precede those that don't.
- `It partition_point(first, last, pred)` – Returns iterator to first element where `pred` is false (range must be partitioned).

Merging (sorted ranges)

- `It merge(f1, l1, f2, l2, dest, comp)` – Merges two sorted ranges into `dest`. Returns iterator past last merged element.
- `void inplace_merge(first, mid, last, comp)` – Merges two consecutive sorted ranges in-place. Returns nothing.

Set Operations (sorted ranges)

Destination must have sufficient space allocated.

- `It set_union(f1, l1, f2, l2, dest, comp)` – Elements in either range. Returns iterator past last written element.
- `It set_intersection(f1, l1, f2, l2, dest, comp)` – Elements in both ranges. Returns iterator past last written element.
- `It set_difference(f1, l1, f2, l2, dest, comp)` – Elements in first but not second. Returns iterator past last written element.
- `It set_symmetric_difference(..., comp)` – Elements in exactly one range. Returns iterator past last written element.
- `bool includes(f1, l1, f2, l2, comp)` – Returns true if second range is a subset of first.

Permutations

- `bool next_permutation(first, last, comp)` – Transforms to next lexicographic permutation. Returns false if wrapped to first permutation.
- `bool prev_permutation(first, last, comp)` – Transforms to previous permutation. Returns false if wrapped to last permutation.

Heap Operations

- `void make_heap(first, last, comp)` – Converts range to max-heap. Returns nothing.
- `void push_heap(first, last, comp)` – Inserts element at `last-1` into heap `[first, last-1)`. Returns nothing.
- `void pop_heap(first, last, comp)` – Moves max element to `last-1`, shrinks heap. Returns nothing.
- `void sort_heap(first, last, comp)` – Sorts a heap into ascending order. Returns nothing.
- `bool is_heap(first, last, comp)` – Returns true if range is a valid max-heap.

4.4. Numeric Algorithms

```
#include <numeric>
```

- `T accumulate(first, last, init, binop)` – Returns `init op a[0] op a[1] op ...` (default `op` is `+`).
- `It partial_sum(first, last, dest, binop)` – Writes prefix sums: `dest[i] = a[0] + ... + a[i]`. Returns iterator past last written element.
- `T inner_product(f1, l1, f2, init, add_op, mult_op)` – Returns `init + a0 · b0 + a1 · b1 + ...`.
- `It adjacent_difference(first, last, dest, binop)` – Writes differences: `dest[i] = a[i] - a[i-1]`. Returns iterator past last written element.
- `void iota(first, last, val)` – Fills with `val, val+1, val+2, ...`. Returns nothing.
- `T reduce(first, last, init, binop)` – Parallel-friendly `accumulate` (C++17). Returns reduced value.
- `It inclusive_scan(first, last, dest, binop, init)` – Parallel-friendly `partial_sum` (C++17). Returns iterator past last written element.
- `It exclusive_scan(first, last, dest, init, binop)` – Like inclusive but excludes current element. Returns iterator past last written element.

4.5. Compare Function

Custom comparator:

```
bool comp(const T& a, const T& b) {  
    return a < b; // true if a should come before b  
}
```

5. Containers

5.1. Common Methods

These methods are common across most STL containers.

Iterators

<code>c.begin()</code>	Returns iterator to the first element.
<code>c.end()</code>	Returns iterator to one past the last element.
<code>c.rbegin()</code>	Returns reverse iterator to the last element.
<code>c.rend()</code>	Returns reverse iterator to one before the first element.
<code>c.cbegin()</code>	Returns const iterator to the first element.
<code>c.cend()</code>	Returns const iterator to one past the last element.

Capacity

<code>c.size()</code>	Returns the number of elements.
<code>c.max_size()</code>	Returns the maximum possible number of elements.
<code>c.empty()</code>	Returns true if the container is empty.
<code>c.capacity()</code>	Current allocated storage (vector, string only).
<code>c.reserve(n)</code>	Reserve storage for at least n elements (vector, string only).
<code>c.shrink_to_fit()</code>	Reduce capacity to fit size (vector, deque, string).

Modifiers

<code>c.clear()</code>	Removes all elements from the container.
<code>c.swap(other)</code>	Swaps contents with another container.

5.2. Sequential Containers

Element Access

	array	vector	deque	list
<code>[]</code>	✓	✓	✓	×
<code>at()</code>	✓	✓	✓	×
<code>front()</code>	✓	✓	✓	✓
<code>back()</code>	✓	✓	✓	✓
<code>data()</code>	✓	✓	×	×

Array

Definition:

`array<T, N> a;`

Fixed-size container. Size must be known at compile time.

Modifiers

<code>a.fill(value)</code>	Fill array with value.
----------------------------	------------------------

Vector

Definition:

`vector<T> v;`

Dynamic array with automatic resizing. Supports all array methods plus:

Additional Modifiers

<code>v.push_back(x)</code>	Add element to the end.
<code>v.pop_back()</code>	Remove last element.
<code>v.emplace_back(args)</code>	Construct element in place at the end.
<code>v.insert(pos, x)</code>	Insert element at iterator position.
<code>v.emplace(pos, args)</code>	Construct element in place at position.
<code>v.erase(pos)</code>	Remove element at iterator position.

Capacity Management

<code>v.capacity()</code>	Current allocated storage.
<code>v.reserve(n)</code>	Reserve storage for n elements.
<code>v.shrink_to_fit()</code>	Reduce capacity to fit size.

`tie` can be used to unpack a vector.

Example: `tie(a, b, c) = v;`

Deque

Definition:

`deque<T> d;`

Double-ended queue. Supports all array element access (except `data()`) and all vector modifiers (except capacity management), plus:

Additional Front Operations

<code>d.push_front(x)</code>	Add element to the front.
<code>d.pop_front()</code>	Remove first element.
<code>d.emplace_front(args)</code>	Construct element in place at the front.

5.3. Associative Containers

Set

Definition

`set<type> s;`
`multiset<type> s;`
`unordered_set<type> s;`
`unordered_multiset<type, hasher> s;`
hasher: Refer Custom Hash Function.

Set operations

<code>s.insert(item);</code>	
<code>s.erase(item);</code>	
<code>s.count(item);</code>	
<code>s.find(item);</code>	Returns the iterator that points to <code>item</code> .
<code>s.insert()</code>	Inserts elements in <code>s</code> .
	Eg: Insert elements from <code>t</code> to <code>s</code> .
	<code>v.insert(v.end(), u.begin(), u.end());</code>
<code>s.lower_bound(x)</code>	Returns iterator to the smallest element that is larger than or equal to <code>x</code> .
<code>s.upper_bound(x)</code>	Returns iterator to the smallest element that is larger than <code>x</code> .

NOTE: erase method remove all instances of the item in the multiset.

Map

`map` : Balanced binary tree.

`unordered_map`: Hash list.

Definition:

`map<type1, type2> m;`
Eg: `map<string, int> m;`
`unordered_map<type1, type2, hasher> m;`
hasher: Refer Custom Hash Function.

Operations on map

<code>m["banana"]</code>	Retrieve value associated with <code>banana</code> .
<code>m.count("banana")</code>	Return 1 if the key is present else 0.

5.4. Container Adapters

Queue

Eg: `queue<int> q;`

<code>q.push(5)</code>	Adds element to the end.
<code>q.pop()</code>	Removes first element.
<code>q.front()</code>	Returns first element.

Stack

```
Eg: stack<int> s;
    s.push(5)
    s.top()
    s.pop()
```

Priority queue

Format:

priority_queue(type, vector<type>, compare)

Max priority queue [Default]

Eg: priority_queue<int> q;

Min priority queue

priority_queue<int, vector<int>, greater<int>> q;

q.push(3)

q.top() Returns the largest element.

q.pop() Removes the largest element.

6. Libraries

6.1. Input output

File Access

Format:

FILE *fp;

FILE *fopen(char *name, char *mode);

int fclose(FILE *fp);

fp = fopen(name, mode);

Allowable modes include:

"r" read.

"w" write.

"a" append.

"b" binary files: "rb", "wb", "ab".

NOTE: **Difference between rb and r etc:**

While using just "r" might work in writing binary files, it might cause trouble due misinterpretation of special characters like LF and CR.

See: <https://stackoverflow.com/q/2174889/5607735>

Standard IO Can be treated in the same way as file pointer.

stdin, stdout, stderr

Reading and writing a file

```
size_t fread(const void* ptr, size_t size,/
size_t count, FILE* stream);
```

```
size_t fwrite(const void* ptr, size_t size,/
size_t count, FILE* stream);
```

Character I/O

getchar Takes input from standard input.

int getchar()

Equivalent togetc(stdin)

putchar Output to standard output.

int putchar(int)

Equivalent toputc(c), stdout

getc int getc(FILE *fp), Eg: getc(stdin)

putc int putc(int c, FILE *fp)

ungetc

Line I/O

gets Reads until EOF.

gets deletes terminal \n

puts Writes line to stdout.

puts adds terminal \n

fgets char *fgets(char *line, int maxline, FILE *fp)

fputs int *fputs(char *line, FILE *fp)

getline Eg: getline(cin, s). Where s is a string.

NOTE: Never use gets

gets does not check for buffer overrun, and keeps reading until it encounters new line or EOF. **It has been used to break computer security.**

printf and scanf

Conversion formats for printf and scanf

Character	Argument type; Printed as
d,i	int; decimal number
o	int; unsigned octal number (without leading zero)
x,X	int: unsigned hexadecimal number (without a leading 0x or 0X), using abcdef or ABCDEF.
u	int; unsigned decimal number.
zu	size_t.
c	int; single character.
s	char *; Print string, scan word. NOTE: scanf reads only a word and not the entire string.
f	double; [-]m.ddddd.
e,E	double; [-]m.ddddd±xx, or [-]m.ddddd±xx.
g,G	double; %e or %E if exponent is < -4 or >= precision, else use %f.
p	void *; pointer.
%	no argument; Print %.

Between % and conversion character there may be in order:

- Minus sign: Left justification.
- Number: Minimum field width.
- Period: Separates field width from precision.
- Number: Precision.
 - For float: number of digits after decimal.
 - For int: minimum number of digits.
 - For string: maximum number of characters to be printed.
- h: for short integer. l: for long integer.

Formatting rules

Examples

%*d

%d

%6d

%6f

%2f

%6.2f

:s:

:%10s:

:.10s:

:-10s:

:.15s:

:-15s:

:%15 .10s:

:-15 .10s:

Wildcard

Here precision can be specified dynamically. Eg: printf("%*d", 6, foo);, this is equivalent to printf("%6d",foo);

Integers

print as decimal integer.

print as decimal integer, at least 6 characters wide.

Floating point numbers

print as floating point.

print as floating point, 2 characters after decimal point.

print as floating point, at least 6 characters wide and 2 characters after decimal point.

String: example hello, world (12 chars).

:hello,_world:

:hello,_world:

:hello,_wor:

:hello,_world:

:hello,_world:

:hello, world_:

:_hello, wor:

:hello,_wor_:

Printf and variants

- int printf(char *format, arg1, arg2, ...);
Print output to stdout.
- int sprintf(char *string, char *format arg1, arg2, ...);
Print output to string.
- int fprintf(FILE *fp, char *format arg1, arg2, ...);
Print output to file pointed by the file pointer fp.

Example:

```
printf("%s\n", string-var);
```

```
printf("%s", "hello, world");
```

```
printf("square of n is %d\n", n_square);
```

Scanf and variants

- int scanf(char *format, arg1, arg2, ...);
Scan input from stdin.
- int sscanf(char *string, char *format arg1, arg2, ...);
Scan input from string.
- int fscanf(FILE *fp, char *format arg1, arg2, ...);
Scan input from the file pointed by the file pointer fp.

NOTE1: unlike printf, in scanf the variable are pointers.

NOTE2: In scanf %s reads a word and not the entire string.

Also see: <https://stackoverflow.com/a/1248017/5607735>

Examples:

```
scanf("%d", &n);
```

```
sscanf(string, "%d", &n);
```

```
fscanf(fp, "%d", &n);
```

6.2. cin and cout

`cin` Read input until space, or tab or LF.
`cout` Output.

NOTE:

- Use the following to increase speed of I/O.
`ios_base::sync_with_stdio(false);`
`cin.tie(nullptr); cout.tie(nullptr);`
- Use `cin.ignore()` after `cin` and before using `getline`.

6.3. Miscellaneous

```
int fflush(FILE *stream)
```

Causes any buffered but unwritten data to be written. The effect is undefined on an input stream.

```
int fclose(FILE *stream)
```

Flushes any unwritten data for `stream`, discards any unread buffered input.

6.4. String

A `string` is like a vector of characters.

Definition:

```
string s = "Hello";
```

String operations:

<code>a + b</code>	Concatenate.
<code>s.append(t)</code>	append <code>t</code> to <code>s</code> .
<code>s.compare(t)</code>	returns 0 if <code>s == t</code> , returns <code>< 0</code> if <code>s < t</code> , else returns <code>> 0</code> .
<code>s.substr(start, len)</code>	Select a substring.
<code>s.find(str2)</code>	find first occurrence of <code>str2</code> .
<code>s.find(str2, pos)</code>	Find first occurrence of <code>str2</code> by searching from the position <code>pos</code> .

<code>x</code>	Hex.
<code>o</code>	Octal.
<code>t</code>	Binary.
<code>d</code>	Decimal.
<code>u</code>	Unsigned decimal.
<code>i</code>	Instruction.
<code>c</code>	Character.
<code>s</code>	String.
<code>b</code>	byte.
<code>h</code>	Halfword.
<code>w</code>	Word.
<code>g</code>	Giant.

7. Devtools

7.1. G++

Usage:

```
g++ option hello.cpp -o hello
```

```
g++ hello.cpp -o hello
```

```
g++ -O3 hello.cpp -o hello
```

Options:

<code>-o</code>	Output file name.
<code>-I</code>	Include additional directories in the search path. Eg: <code>g++ -I /home/user/libs foo.cpp</code>
<code>-O<level></code>	Optimization level, starts from 0. Eg: <code>-O3</code>
<code>-g</code>	Creates breakpoints for debugging using GDB.
<code>-lm</code>	Missing link. Used to link certain libraries like <code>math.h</code> .
<code>-E</code>	Output preprocessed but uncompiled code.
<code>-S</code>	Compile but do not assemble.
<code>-c</code>	Compile and assembly, but do not link.

7.2. GDB

To run: `gdb hello`

<code>break</code>	Set break point. Eg: <code>break function</code> or <code>break line number</code> .
<code>run</code>	Run the program. The program stops at every break point.
<code>next</code>	Run until next breakpoint.
<code>print</code>	Eg: <code>print i</code> or <code>print &i</code>
<code>sizeof</code>	Eg: <code>print sizeof(i)</code>
<code>&i</code>	Address of <code>i</code> .
<code>*j</code>	Content of memory location <code>j</code> .
<code>ptype</code>	get type of a variable. Eg: <code>ptype(i)</code>
<code>set var</code>	Reassign variable. Eg: <code>set var i = 1</code>
<code>disassemble</code>	Use after break and run. Gives assembly code.

Accessing memory location using `x`

Usage: `x/nfs. nfs` describes the format.

`n` - Number of units to display.

`f` - Number format.

`s` - Size of each unit.